

VINUNIVERSITY

Circuit Breakers, Caching & Reliability

AICB-P2T3 · Ngày 25 · Chương 5 — Agent Production-Ready

Giảng viên

VinUniversity · Phase 2 · Track 3 · Tuần 5

HÃY SUY NGHĨ...

“Khi LLM provider timeout trong production — agent của bạn tự phục hồi, hay kéo sập cả workflow?”

Giữ câu hỏi này trong đầu khi học bài hôm nay

Nội dung bài học

1. Mở đầu: Reliability Stack
2. Failure Modes
3. Circuit Breaker
4. Fallback & LLM Gateway
5. Caching & Cost Budgeting
6. Observability, SLO & Chaos
7. Live Demo & Lab 25

Mở đầu: Reliability Stack

01

Mục tiêu, timeline và 3 lớp của một agent production-ready: Resilience · Efficiency · Observability

Sau buổi học, học viên làm được gì?

Conceptual outcomes

- Nhận diện **6 nhóm lỗi** production của LLM agent
- Giải thích vì sao **retry sai cách** biến lỗi nhỏ thành outage
- Giải thích 3 trạng thái circuit breaker & trip logic
- Phân biệt prompt cache, semantic cache, tool cache
- Thiết kế SLI / SLO / SLA + error budget cho agent

Practical outcomes (Lab 25B)

- Thay client gây retry storm: phân loại lỗi, retry 1 tầng có budget
- Breaker + fallback ladder qua **6 băng sự cố**
- Semantic cache có scope + stale-if-error
- Workflow exactly-once (idempotency + checkpoint)
- Burn-rate alert, error budget, postmortem

Deliverable hôm nay

Client PASS **6 băng sự cố + băng cache C1**, alert theo **SLO 99,5%**, postmortem G2 + nộp **Milestone 2** (fine-tuned model + guardrails + eval pipeline + reliability patterns).

Timeline buổi học: 2h lý thuyết + 2h lab



Lý thuyết có tương tác (120')

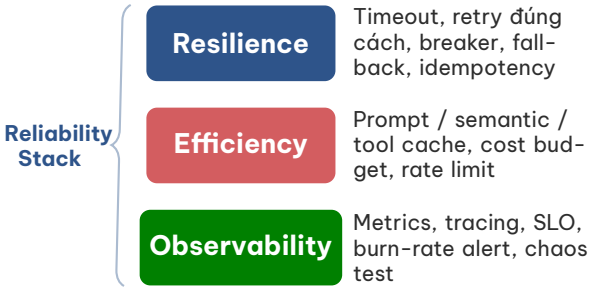
- Cuối mỗi phần: **Checkpoint** 3 câu, vote A/B/C/D (∼1 phút/câu)
- 3 hoạt động nhóm: cascading failure (5'), fallback policy (8'), cache hay không (5')
- **Live demo** 10': provider sập giữa chừng (simulator Lab 25B)

Lab 25B — Game Day (thiết kế 4h)

- 2h đầu: kickoff (hình G2 + D4) + Part A–C: phân loại lỗi, retry, breaker, fallback → 6 băng sự cố
- 2h sau: G cache & stale-if-error, D idempotency, E burn-rate alert, F report + postmortem
- Nộp kèm **Milestone 2**

Slide có nhãn ***THAM KHẢO*** *có thể lướt nhanh khi thiếu giờ — học viên tự đọc*

Reliability Stack — 3 lớp



Vì sao reliability là bắt buộc?

- >70% tổ chức dùng ≥ 3 model; nhóm dùng > 6 model gần gấp đôi sau 1 năm
- 37% doanh nghiệp dùng ≥ 5 model (2024: 29%)
- 02/2026: **5%** LLM call lỗi — **60%** trong số đó do vượt rate limit

Analogy

Circuit breaker giống **aptomat điện**: ngắt khi quá tải, chỉ đóng lại khi đã an toàn.

Nguồn: Datadog, State of AI Engineering (04/2026); a16z, How 100 Enterprise CIOs Are Building and Buying Gen AI in 2025 (06/2025).

Failure Modes

02

Reliability bắt đầu từ việc gọi đúng tên lỗi: transient, degraded, outage, orchestration, tool, business action

Production agent fail như thế nào? — 6 nhóm lỗi

#	Nhóm lỗi	Triệu chứng	Phản ứng chính
1	Provider transient	429 rate limit, 500 / 529 overloaded, timeout	Retry 1 tầng + backoff + jitter, tôn trọng `retry-after`
2	Degraded latency	P95 / TTFT tăng mạnh, chưa báo lỗi	Timeout rõ ràng + circuit breaker (tính cả call chậm)
3	Full outage	Provider / region không phản hồi	Fallback: region / tier khác → provider khác
4	Orchestration	Loop vô hạn, state sai, context phình	Max steps / budget, checkpoint, retry theo node
5	Tool / cache	Schema lệch, auth hết hạn, dữ liệu stale	Validate I/O, TTL, breaker riêng cho tool
6	Business action	Side effect lỗi hoặc bị chạy 2 lần	Idempotency key, compensation, HITL

Lưu ý: Lỗi có thể nằm ở provider, gateway, cache, tool hay business action — reliability là **system property**, không phải "thêm retry".

Retry đúng cách: lỗi transient, backoff + jitter, 1 tầng

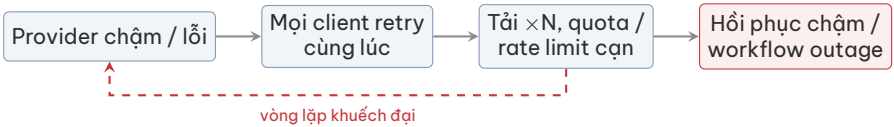
```python
RETRYABLE = {408, 429, 500, 502, 503, 504, 529}

def call_with_retry(fn, attempts=3, base=0.5, cap=8.0):
    for i in range(attempts):
        try:
            return fn()
        except ProviderError as e:  # hypothetical
            if (e.status not in RETRYABLE
                    or e.quota_exhausted  # no credit
                    or i == attempts - 1):
                raise         # never retry 400/401/403
            # retry-after first, else exponential
            # backoff with FULL jitter
            delay = e.retry_after or random.uniform(
                0, min(cap, base * 2 ** i))
            time.sleep(delay)
```

- **Chỉ retry lỗi transient**: 408, 429, 5xx, 529, timeout. Không retry 400 / 401 / 403 / 422.
- **429 do hết credit / spend limit**: không có `retry-after` → retry vô ích, alert ngay.
- **Jitter**: tránh mọi client retry cùng một nhịp (thundering herd).
- **1 tầng**: Python SDK của Anthropic & OpenAI mặc định đã retry **2 lần**, timeout **10 phút**.
- **Retry budget**: chỉ retry khi tỷ lệ retry $< 10\%$ request.

Nguồn: Brooker, "Exponential Backoff And Jitter" (AWS, 2015); Google SRE Book, "Handling Overload"; docs SDK Anthropic / OpenAI (10/2026).

Case thật: retry làm sự cố tệ hơn



Google Cloud – 12/06/2025

Policy có trường rỗng làm Service Control crash toàn cầu. Khi restart ở us-central1 xảy ra *herd effect*: "*Service Control did not have the appropriate randomized exponential backoff implemented*" → region này mất ~2h40' mới hồi phục. Lan sang Cloudflare: Workers AI lỗi 100%, AI Gateway lỗi đỉnh 97%.

OpenAI – 03/02/2026

"*Increased retry traffic during the disruption amplified load on downstream systems, which slowed recovery in one region.*" Biện pháp được công bố: **thêm circuit breaker** để ngăn retry amplification trong sự cố cục bộ.

Nguồn: status.cloud.google.com/incidents/ow5i3PPK96RduMcb1SsW; blog.cloudflare.com (12/06/2025); status.openai.com – write-up sự cố 03/02/2026.

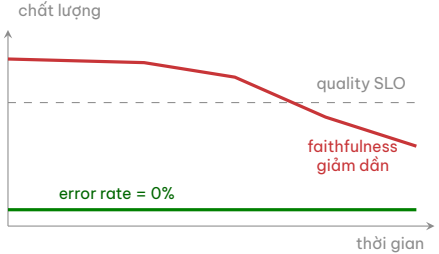
Thảo luận nhóm — 5 phút

Chọn 1 sản phẩm agent bạn biết (CSKH, coding agent, trợ lý ngân hàng…). Provider chính **timeout 30 giây** liên tục trong 15 phút.

- Nếu **chưa có** reliability layer: user thấy gì? Phần còn lại của hệ thống bị ảnh hưởng ra sao?
- Đề xuất 1 biện pháp cho mỗi bước **contain** → **isolate** → **recover**.
- Metric nào chứng minh biện pháp có hiệu quả?

Nhóm 2–3 người · mời 1–2 nhóm chia sẻ

Silent degradation: không lỗi nhưng chất lượng giảm



- Provider cập nhật model / alias trỏ sang bản mới
- Prompt / schema đổi nhưng eval không đổi
- Knowledge base stale, retrieval yếu dần
- Cache trả câu "đúng hôm qua, sai hôm nay"

Case thật — Anthropic (09/2025)

3 bug hạ tầng chồng lấn (08–09/2025): routing sai context window (đỉnh 16% request Sonnet 4 trong 1 giờ), lỗi output trên TPU (chèn ký tự Thái / Trung vào câu tiếng Anh), XLA miscompile top-k.

"*The evaluations we ran simply didn't capture the degradation users were reporting.*"

Bài học họ rút ra: chạy eval **liên tục trên hệ thống production thật**.

Lưu ý: Quality SLO phải đi cùng uptime SLO: eval / LLM-as-judge trên mẫu traffic thật (nối Ngày 24 — RAGAS). Error rate = 0% không đủ.

Nguồn: anthropic.com/engineering/a-postmortem-of-three-recent-issues

Checkpoint 1 — Failure Modes

CÂU 1/3

Log 1 tuần của agent: **5%** LLM call bị lỗi. Theo dữ liệu Datadog (02/2026), nguyên nhân **phổ biến nhất** là gì, và phản ứng đầu tiên phù hợp?

A. Provider sập hoàn toàn → chuyển ngay mọi request sang provider khác

B. Vượt rate limit (429) → backoff có jitter, tôn trọng `retry-after`, giới hạn concurrency / token phía client

C. Model hallucinate → giảm temperature

D. Mạng phía user chập chờn → bỏ qua, không cần xử lý

Cả lớp vote A / B / C / D — 1 phút

Provider trả **529 overloaded** cho mọi request. Một lượt user đi qua 3 tầng: app retry tối đa 3 lần, gateway `num_retries=2`, SDK để mặc định (`max_retries=2`). Lượt đó tạo **tối đa** bao nhiêu request tới provider?

A. 3

B. 7

C. 12

D. 36

Cả lớp vote A / B / C / D — 1 phút

Dashboard: error rate 0%, P95 ổn định — nhưng khiếu nại "trả lời sai" tăng 3 lần trong 2 tuần. Biện pháp nào phát hiện loại sự cố này **sớm nhất**?

A. Thêm retry cho mọi request

B. Tăng timeout của gateway

C. Quality SLO: chạy eval / LLM-as-judge liên tục trên mẫu traffic production thật, alert khi chất lượng drift

D. Chuyển hẳn sang provider khác

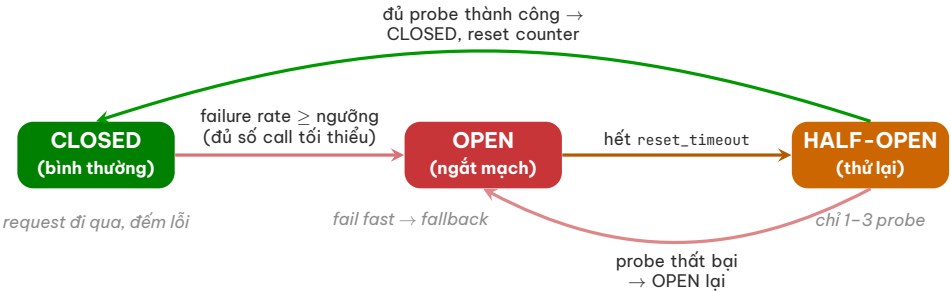
Cả lớp vote A / B / C / D — 1 phút

Circuit Breaker

03

Ngắt gọi dependency đang hỏng để fail fast; tự probe để phục hồi

Circuit Breaker: 3 trạng thái



Isolation

Breaker theo **provider** × **model** (× **region**). Provider A mở không được kéo provider B hay cache layer sập theo.

Fail fast có lợi gì?

Không giữ worker chờ timeout, không dội thêm tải vào dependency đang hỏng, user nhận fallback ngay.

Trip logic: đếm lỗi gì, trong cửa sổ nào?

Lỗi	Tính vào breaker?
Timeout, connection error	**Có**
5xx, 529 overloaded	**Có**
429 rate limit (có `retry-after`)	Không — provider vẫn khỏe: cooldown theo `retry-after`, route deployment khác
429 hết credit / spend limit	Không — alert ngay, breaker không giúp
400 / 401 / 403 / 422	**Không** — lỗi của mình, không phải provider
Output sai schema, bị content filter	Không — validation / fallback riêng

Cửa sổ đếm

- **Count-based**: "5 lỗi liên tiếp" — đơn giản, nhưng nhạy khi traffic thấp
- **Rate-based**: "$\geq$ 50% lỗi trong 20 call gần nhất, tối thiểu 10 call" — ổn định hơn
- **Slow call**: call vượt latency SLO cũng tính là lỗi → bắt được *degraded*

```yaml
breaker:  # Lab 25B
  window_size: 20  # calls
  min_calls: 10
  failure_rate_threshold: 0.5
  slow_call_s: 5.0  # G4: +5 s
  slow_rate_threshold: 0.6
  open_duration_s: 10
  half_open_max_probes: 2
```

Nguồn: Khối `breaker:` *trong* `configs/gameday.yaml` *của Lab 25B. Resilience4j:* `slidingWindowSize`, `minimumNumberOfCalls`, `slowCallDurationThreshold`, `waitDurationInOpenState`; *pybreaker:* `fail_max`, `reset_timeout`, `success_threshold`, `exclude`.

Circuit breaker ở mức code

```python
class CircuitBreaker:       # one per provider x model
    def call(self, fn, *args, **kwargs):
        if self.state == "OPEN":
            if self.elapsed() < self.reset_timeout:
                raise CircuitOpen(self.name)  # fail fast
            self.state, self.probes = "HALF_OPEN", 0
        if self.state == "HALF_OPEN":
            if self.probes >= self.max_probes:
                raise CircuitOpen(self.name)  # limit probes
            self.probes += 1
        t0 = time.monotonic()
        try:
            result = fn(*args, **kwargs)
        except HEALTH_ERRORS:      # timeout/5xx/529
            self.record_failure()  # HALF_OPEN -> OPEN
            raise                  # 429, 4xx: not counted
        self.record_success(time.monotonic() - t0)
        return result  # slow = bad; N ok -> CLOSED
```

Những điểm hay sai

- `except Exception`: 400 / 401 mở breaker oan → **chỉ đếm timeout, 5xx, 529** (429: cooldown riêng)
- Bỏ qua call chậm → không bắt được *degraded*
- HALF-OPEN không **giới hạn probe**
- Nhiều instance, state in-memory → **Redis** / breaker ở gateway
- Async: cần **lock**; bỏ qua kết quả về muộn (permit / generation)

Thư viện Python (10/2026)

- **pybreaker** 1.4: sync, Redis storage; async cần Tornado
- **circuitbreaker** 2.1: hỗ trợ coroutine
- **purgatory**: asyncio + Redis
- Gateway: LiteLLM cooldown, Portkey

Half-open probe: request thật hay health check?

THAM KHẢO

VINUNIVERSITY

Real request (đơn giản)

- Một request thật của user đi qua làm probe
- Đo đúng latency & chất lượng thực tế
- **Rủi ro**: side effect nếu request có ghi / thanh toán; user đó chịu rủi ro lỗi

Health check / canary (an toàn)

- Prompt canary cố định, `max_tokens` nhỏ, không side effect
- Không ảnh hưởng user thật
- **Rủi ro**: canary OK nhưng request thật vẫn lỗi (prompt dài, tool calling, model khác)

Khuyến nghị

Probe bằng request **read-only** giống traffic thật (cùng model, cùng tính năng như tool calling), giới hạn 1–3 probe; chỉ đóng breaker sau **nhiều** lần thành công liên tiếp nếu dependency hay chập chờn.

Timeout · Bulkhead · Hedging — 3 người bạn của breaker

THAM KHẢO VINUNIVERSITY

Timeout

SDK Python của Anthropic & OpenAI mặc định timeout **10 phút**. Đặt rõ:

- connect ~5s
- time-to-first-token (stream) 5–10s
- tổng thời gian theo SLO

Timeout là tín hiệu *degraded* cho breaker.

Bulkhead

Giới hạn concurrency / queue **riêng** cho từng provider & tool (semaphore, pool riêng).

Provider A treo không giữ hết worker của provider B — và của các tính năng không dùng LLM.

Hedging

Gửi request dự phòng khi request đầu chậm hơn p95. Google: p99,9 giảm từ 1.800 ms còn 74 ms với chỉ +2% request.

Với LLM: tốn gấp đôi token → chỉ dùng cho request ngắn / rẻ, hoặc hedge sang model nhỏ.

Nguồn: Dean & Barroso, "The Tail at Scale", CACM 56(2), 2013; Nygard, Release It! (2nd ed.) — Bulkhead pattern.

Checkpoint 2 — Circuit Breaker

CÂU 1/3

Bug deploy làm app gửi request **thiếu field bắt buộc** → provider A trả **400** liên tục. Breaker của A đặt `fail_threshold=5`. Hành vi đúng là gì?

A. Breaker mở → tự chuyển sang provider B

B. Retry 400 với exponential backoff tới khi thành công

C. Không tính 400 vào breaker: trả lỗi + alert để sửa code — fallback sang B cũng sẽ lỗi y hệt

D. Tăng `fail_threshold` lên 50 cho an toàn

Cả lớp vote A / B / C / D — 1 phút

Checkpoint 2 — Circuit Breaker

CÂU 2/3

Gateway chạy **8 instance**, mỗi instance giữ breaker **in-memory** (`fail_threshold=5`). Provider A sập. Hệ quả nào **đúng**?

A. Cần tới $\sim$40 request lỗi (8×5) trước khi mọi instance mở; mỗi instance tự half-open & probe riêng $\rightarrow$ trạng thái không nhất quán

B. Không vấn đề gì — mỗi instance tự bảo vệ là đủ

C. Breaker của provider B cũng mở theo

D. Nên bỏ breaker, chỉ cần retry

Cả lớp vote A / B / C / D — 1 phút

Breaker vừa chuyển sang **HALF-OPEN**. Cùng lúc có **200 request** đến. Thiết kế đúng là gì?

A. Cho cả 200 request đi qua để kiểm tra nhanh

B. Đóng breaker ngay vì đã hết `reset_timeout`

C. Chặn cả 200 request và trả HTTP 503 cho user

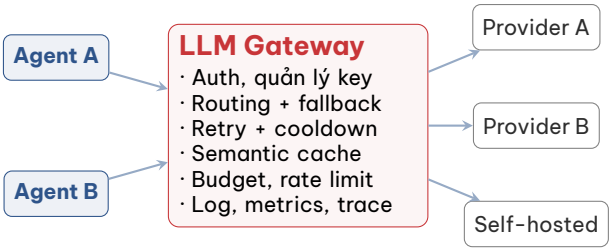
D. Chỉ cho 1 (hoặc vài) probe đi qua; phần còn lại tiếp tục fail fast sang fallback tới khi probe thành công

Cả lớp vote A / B / C / D — 1 phút

Fallback & LLM Gateway

Fallback chain giữ trải nghiệm ở mức chấp nhận được; gateway gom routing, retry, cache và budget về một chỗ

LLM Gateway: 1 API, nhiều provider

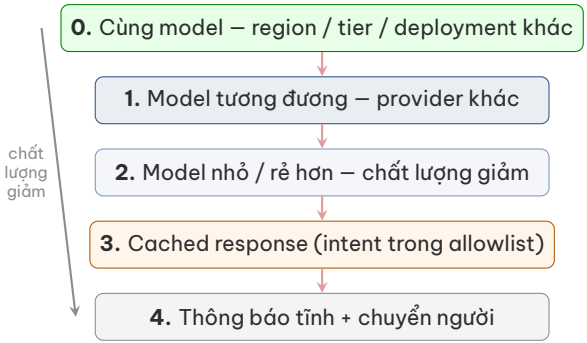


Bức tranh 10/2026

- **LiteLLM** (OSS, SDK + proxy): fallbacks, cooldown, budget, semantic cache
- **Portkey**: open-source toàn bộ gateway từ 03/2026; có circuit breaker
- **Kong AI Gateway**, **Envoy AI Gateway** (đổi tên *Agent Router*, 09/2026): tầng hạ tầng, token rate limit
- **Cloudflare / Vercel AI Gateway**, **OpenRouter**: managed; cache Cloudflare chỉ exact-match
- **Helicone**: về Mintlify (03/2026), chế độ bảo trì → rủi ro vendor

Lưu ý: Gateway là **single point of failure mới**: chạy ≥ 2 replica, health check, và có đường bypass khẩn cấp.

Fallback ladder: graceful degradation



Bậc 0 không cần eval lại: Bedrock cross-region inference, Azure spillover (PTU → standard), service tier khác của OpenAI / Gemini.

Compatibility check

- Tool calling, structured output / JSON mode
- Context length, multimodal, tiếng Việt
- Safety / policy behavior, latency & cost
- **Chạy cùng eval suite** trên prompt của task

3 kiểu fallback

- **Quality**: best → second best
- **Latency**: model đang nhanh nhất
- **Cost**: rẻ nhất mà vẫn đủ tốt (budget-aware)

Theo task class

Coding fallback khác chat; task rủi ro cao (y tế, tài chính) **không** xuống model yếu — chuyển người.

LiteLLM Router: fallback + cooldown + retry

```python
from litellm import Router

def dep(group, model, **kw):          # one deployment
    return {"model_name": group,
            "litellm_params": {"model": model, **kw}}
router = Router(
    model_list=[
        dep("primary", "openai/<model-A>", rpm=500),
        dep("primary", "azure/<deploy-A>", api_base=BASE),
        dep("backup", "anthropic/<model-B>"),
        dep("cheap", "gemini/<model-C>"),
    ],
    num_retries=2, timeout=30,         # retry in ONE layer
    allowed_fails=3, cooldown_time=30, # ~ circuit breaker
    fallbacks=[{"primary": ["backup", "cheap"]}],
    context_window_fallbacks=[{"primary": ["backup"]}],
)
resp = router.completion(model="primary", messages=msgs)
```

- **Model group** (`primary`): nhiều deployment, load balance theo `routing_strategy` (shuffle, latency, usage, cost)
- `allowed_fails` + `cooldown_time`: deployment lỗi quá ngưỡng/phút bị tạm loại — tương đương breaker
- `fallbacks`: ladder giữa các group; lỗi tràn context dùng `context_window_fallbacks`, lỗi policy dùng `content_policy_fallbacks`
- `RetryPolicy` / `AllowedFailsPolicy`: số lần theo **loại lỗi**
- Test: `mock_testing_fallbacks=True` (Router Python)

Kiểm tra trên LiteLLM 1.105 (10/2026). Tên model để dạng <...>: điền model hiện hành.

Gateway cũng là rủi ro: sự cố LiteLLM 03/2026

THAM KHẢO

VINUNIVERSITY

Chuyện gì đã xảy ra? (24/03/2026)

- 2 bản PyPI **1.82.7** và **1.82.8** bị chèn mã đánh cắp credential: env vars, SSH, cloud, Kubernetes, DB
- 1.82.8 thêm file `.pth` → chạy mỗi khi Python khởi động, kể cả khi không import litellm
- Nghi do token PyPI lộ qua chuỗi tấn công TeamPCP / Trivy (đường lây chính xác còn tranh luận)
- Docker image chính thức (pin dependency) **không** bị ảnh hưởng; bản sạch: 1.83.0

Bài học cho reliability

- Gateway giữ **mọi** API key → mục tiêu giá trị cao
- Sự cố bảo mật cũng là sự cố availability: rotate toàn bộ key = downtime
- Pin version + hash: lockfile, `pip install --require-hashes`
- Dùng image đã kiểm định; không rebuild "latest" hằng ngày
- Tối thiểu secret trong runtime; egress allowlist
- Có runbook rotate key trong < 1 giờ

Nguồn: docs.litellm.ai/blog/security-update-march-2026 (cập nhật 30/03/2026).

Action có side effect: idempotency & durable execution

Vấn đề: retry / fallback một tool call "tạo ticket", "chuyển tiền" → **chạy 2 lần**.

- **Idempotency key**: tạo **một lần cho mỗi action** (vd. `session:step`, hoặc UUID lưu trong state) rồi **dùng lại** cho mọi retry / lần chạy lại — `uuid4()` mới mỗi lần gọi thì vô dụng. Server lưu kết quả lần đầu, gặp key cũ thì trả lại (Stripe: header `Idempotency-Key`, giữ ≥ 24 giờ)
- **Durable execution**: checkpoint **sau khi** mỗi bước thành công → resume, không chạy lại bước đã xong (LangGraph checkpointer; Temporal + OpenAI Agents SDK)
- **Compensation (saga)**: bước sau lỗi → hành động bù (hủy ticket, hoàn tiền)
- Action không đảo ngược được → **HITL** (Ngày 27)

```python
from langgraph.types import RetryPolicy

def create_ticket(state):
    key = state["action_id"]   # stable id
    return helpdesk.post(
        "/tickets", json=state["ticket"],
        headers={"Idempotency-Key": key})

builder.add_node(
    "create_ticket", create_ticket,
    retry_policy=RetryPolicy(max_attempts=3))
graph = builder.compile(checkpointer=checkpointer)
graph.invoke(state, config, durability="sync")
```

LangGraph 1.2: `RetryPolicy` *mặc định retry lỗi kết nối & HTTP 5xx, không retry* `ValueError`.

Thảo luận nhóm — 8 phút

Nhóm 3 người. Chọn 1 task: **customer support**, **code review**, **medical triage**, hoặc **HR chatbot nội bộ**. Thiết kế fallback ladder 4 bậc:

- Điều kiện chuyển bậc (breaker open, timeout, budget cạn, context overflow…) và metric kiểm chứng
- Rủi ro lớn nhất mỗi bậc: quality vs latency, cost vs safety, cache vs freshness, static vs user trust
- Task nào **không được** fallback sang model yếu hơn? Vì sao?

Nhóm 2–3 người · mời 1–2 nhóm chia sẻ

Agent đặt lịch dùng **tool calling + structured output**. Primary down; model fallback rẻ nhất **không hỗ trợ tool calling**. Nên làm gì?

A. Vẫn fallback sang model rẻ, hy vọng nó tự trả JSON đúng

B. Retry primary tới khi thành công

C. Tắt vĩnh viễn tính năng đặt lịch

D. Chỉ fallback tới model đã qua compatibility check + eval cho task này; hết bậc → cache / thông báo tĩnh, chế độ "chỉ đọc", báo user

Cả lớp vote A / B / C / D — 1 phút

Model chính ở region `us-east` trả **529** hàng loạt. Thứ tự fallback nào hợp lý nhất cho trợ lý CSKH?

A. Cùng model ở region / deployment / tier khác → model tương đương của provider khác → model nhỏ hơn → cached answer → thông báo tĩnh + chuyển người

B. Model nhỏ cùng provider → thông báo tĩnh

C. Thông báo tĩnh ngay để an toàn

D. Trả cached answer cho mọi câu, rồi mới thử model khác

Cả lớp vote A / B / C / D — 1 phút

Dockerfile của team chạy `pip install litellm` (không pin version) và rebuild hằng ngày. Bài học từ sự cố supply chain **24/03/2026** là gì?

A. Không sao — open source nên an toàn

B. Luôn cài bản mới nhất để nhận bản vá sớm

C. Pin version + hash (lockfile, `--require-hashes`), dùng image đã kiểm định, tối thiểu secret trong runtime, có quy trình rotate key nhanh

D. Bỏ gateway, gọi thẳng từng provider

Cả lớp vote A / B / C / D — 1 phút

05

Caching & Cost Budgeting

Cache đúng chỗ giảm latency & cost; cache sai chỗ tạo câu trả lời stale và rò rỉ dữ liệu giữa user

3 tầng cache cho LLM applications

	Khớp theo	Rủi ro
1. Prompt / prefix cache (provider)	Prefix **chính xác** của prompt	Thấp: output không đổi, chỉ giảm cost / latency
2. Semantic response cache (app)	Embedding similarity $\geq$ ngưỡng	Cao: false hit, stale, rò rỉ giữa user
3. Tool / result cache (API, DB)	Tool + tham số (+ user)	Stale $\rightarrow$ TTL theo độ tươi dữ liệu

Nguyên tắc

Cache cái **deterministic & low-risk trước**. Thứ tự lookup: exact cache $\rightarrow$ semantic cache $\rightarrow$ LLM (vẫn hưởng prompt cache). Semantic cache cần threshold, TTL, invalidation và allowlist theo task.

Prompt caching 2026: provider làm gì cho bạn?

THAM KHẢO

VINUNIVERSITY

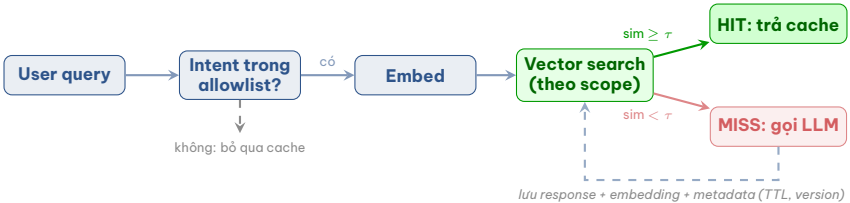
	Anthropic Claude	OpenAI	Google Gemini
Cách bật	`cache_control` (≤ 4 breakpoint); automatic caching từ 02/2026	Tự động	Implicit mặc định (từ 2.5); explicit `CachedContent`
Giá đọc cache	0,1× input (vài model mới thấp hơn)	giảm 50% (GPT-4o) → 90% (GPT-5.x)	≈ 0,1× input
Phí ghi / lưu	ghi 1,25× (TTL 5') hoặc 2× (TTL 1h)	tùy model	explicit: phí lưu trữ theo giờ
Ngưỡng tối thiểu	512–4.096 tokens tùy model	1.024 tokens	2.048–4.096 tokens
Thời gian sống	5 phút (mặc định) hoặc 1 giờ	in-memory 5–10', hoặc giữ 24h	explicit: mặc định 1 giờ
Field để đo	`cache_read_input_tokens`	`cached_tokens`	`cached_content_token_count`

Static trước, dynamic sau

Instructions, tool definitions, tài liệu → rồi mới tới thời gian, user input; đừng chèn timestamp / UUID vào đầu prompt. Chỉ **28%** LLM call dùng cached input (Datadog 2026). Với Claude, token đọc cache **không tính vào ITPM** (đa số model) → caching còn tăng throughput.

Nguồn: docs prompt caching & pricing của Anthropic, OpenAI, Google — kiểm tra 12/10/2026; giá đổi thường xuyên, xem lại trước khi dùng số.

Semantic cache flow



Key & scope

Tách namespace theo **tenant**, model, prompt version, ngôn ngữ. Câu hỏi về dữ liệu **cá nhân / real-time** (đơn hàng, số dư) không vào allowlist, kể cả khi tách theo user. Lưu metadata (nguồn, version) để invalidate khi tài liệu đổi.

Lưu ý: **Cache poisoning / false hit**: "hủy đơn #123" ≈ "hủy đơn #124". Metric bắt buộc: **hit rate** *và* **false-hit rate**. Chú ý chiều ngưỡng: similarity (cao = chặt) vs distance (thấp = chặt).

Semantic cache: hit rate thực tế & false hit

5–60%

hit rate tùy workload: 40–60% nhóm lặp nhiều, 5–15% nhóm ít lặp

19–99%

false-positive khi dùng ngưỡng cố định 0,7 — tùy embedding model

3,8%

false-positive @ 77,6% hit sau khi tune + pre-load cache + lọc chất lượng

Vì sao ngưỡng cố định là chưa đủ?

vCache (ICLR 2026): ngưỡng tĩnh (vd. "cosine > 0,95") không đảm bảo đúng → học ngưỡng riêng cho từng entry theo error bound do mình đặt. Ngưỡng phải tune theo **loại câu hỏi** trên dữ liệu thật.

Công cụ 2026

RedisVL `SemanticCache`; Redis LangCache (managed, preview); LiteLLM `redis-semantic` / `qdrant-semantic`; Kong, Portkey. **GPTCache** (outline gốc): bản cuối 08/2024, không thêm API / model mới.

Nguồn: arXiv 2510.26835 (category-aware caching); InfoQ 11/2025 (FAQ ngân hàng tổng hợp, 1.000 query); arXiv 2502.03771 (vCache).

Thảo luận nhóm — 5 phút

Với mỗi dòng, giơ tay: **cache** / **không cache** / **cache có điều kiện**. Nếu cache: ở tầng nào (prompt / semantic / tool), TTL hay invalidation gì?

Use case	Đặc điểm dữ liệu
FAQ tuyển sinh	lặp lại nhiều, thay đổi theo mùa tuyển sinh
Số dư tài khoản	cá nhân, real-time
Giải thích đoạn code	phụ thuộc chính đoạn code & phiên bản thư viện
Thời tiết hôm nay	đổi theo giờ và địa điểm
Tóm tắt chính sách	ổn định, đổi theo phiên bản văn bản

Nhóm 2–3 người · mời 1–2 nhóm chia sẻ

Cost budgeting: reliability của ví tiền

3 lớp kiểm soát

1. **Per-request cap**: `max_tokens`, số tool call / agent step tối đa, timeout
2. **Per-user / app rate limit**: token bucket tính bằng **LLM tokens**, không chỉ số request
3. **Budget tháng**: cảnh báo 80%; 100% → route model rẻ hoặc dừng (tùy task)

$$\textbf{Cost / call} = (\text{input} - \text{cached}) \cdot p_{in} + \text{cached} \cdot p_{read} + \text{cache_write} \cdot p_{write} + \text{output} \cdot p_{out}$$

LiteLLM virtual key: `max_budget`, `budget_duration`, `tpm_limit`, `rpm_limit`

Metric cần log cho mỗi call

- provider, model, **route reason**
- input / cached / output tokens, cost ước tính
- cache hit / miss, similarity score
- latency, status, circuit state
- feature, tenant (không gắn làm label Prometheus)

Tối ưu đúng chỗ

Đừng chỉ nhìn tổng cost/ngày — cần cost theo **feature** × **model** × **tenant** để tìm đường call đắt: cache trước, đổi model sau.

System prompt 6.000 tokens dùng chung cho mọi request, nhưng dòng **đầu tiên** là `Thời gian: 2026-10-12 09:31:07`. Log cho thấy `cached_tokens` luôn bằng 0. Vì sao?

A. Prompt quá ngắn nên không đủ điều kiện cache

B. Prompt caching khớp theo **prefix chính xác**; timestamp ở đầu đổi mỗi giây → vỡ cả prefix. Đưa phần tĩnh lên đầu, phần động xuống cuối

C. Provider tắt cache vào giờ cao điểm

D. Phải dùng semantic cache mới có cache hit

Cả lớp vote A / B / C / D — 1 phút

Team chuyển semantic cache từ LiteLLM (`similarity_threshold=0.9`) sang RedisVL và đặt `distance_threshold=0.9`. Điều gì xảy ra?

A. Hành vi giữ nguyên

B. Cache gần như không bao giờ hit

C. RedisVL báo lỗi cấu hình

D. Cache trở nên rất "dễ dãi": cosine distance 0,9 ≈ similarity 0,1 → false hit tràn lan

Cả lớp vote A / B / C / D — 1 phút

10.000 request/ngày; mỗi request 20.000 input tokens (18.000 là prefix tĩnh) + 500 output. Giá input $3 / 1M tokens; cache đọc 0,1×, ghi 1,25×. 95% request hit cache prefix, 5% phải ghi lại. Chi phí **input** mỗi ngày xấp xỉ?

A. ≈ $145 (giảm ~76% so với $600)

B. ≈ $570 (giảm 5%)

C. ≈ $60 (giảm 90%)

D. ≈ $750 (tăng vì phí ghi cache)

Cả lớp vote A / B / C / D — 1 phút

Observability, SLO & Chaos

Không đo thì không biết hệ thống đang tốt, chậm, đắt — hay đang trả lời sai

06

SLI, SLO, SLA cho LLM agent

Khái niệm	Ý nghĩa	Ví dụ cho agent
SLI	Metric đo được	% lượt user thành công (kể cả nhờ fallback), P95 latency / TTFT, fallback success, false-hit rate, faithfulness
SLO	Mục tiêu nội bộ	availability $\geq$ 99,5% / 30 ngày (Lab 25B, Part E); TTFT P95 $<$ 2,5s; fallback success $\geq$ 95%; faithfulness $\geq$ 0,85
SLA	Cam kết ngoài, kèm đền bù — **lỏng hơn SLO**	99,0% / tháng cho API khách hàng
Error budget	$1 -$ SLO	0,5% = 3,6 giờ / 30 ngày, hay 5.000 lượt lỗi / 1 triệu lượt

Lưu ý: LLM agent cần thêm **quality SLO**: faithfulness, safety pass rate, escalation correctness — uptime 100% vẫn có thể trả lời sai.

Alert theo burn rate, không theo từng lỗi

Burn rate — tốc độ tiêu error budget = error rate / (1 − SLO). Burn 1 = vừa hết budget đúng cuối kỳ.

Budget	Cửa sổ dài	ngắn	Burn	Xử lý
2%	1 giờ	5 phút	14,4	Page
5%	6 giờ	30 phút	6	Page
10%	3 ngày	6 giờ	1	Ticket

Nguồn: Google SRE Workbook, ch. 5 "Alerting on SLOs" (cửa sổ SLO 30 ngày).

Ví dụ với SLO 99,5%

Burn 14,4 ↔ error rate **7,2%**; burn 6 ↔ 3%; burn 1 ↔ 0,5%.

Cửa sổ dài: lỗi đủ đáng kể. Cửa sổ ngắn: lỗi *vẫn đang* xảy ra → alert tự tắt khi đã hồi phục.

Dashboard đèn giao thông

Mỗi provider × metric (availability, P95, cost / lượt, false-hit, faithfulness): **xanh** trong SLO, **vàng** burn > 1, **đỏ** đang page.

Hết error budget → đóng băng release rủi ro, ưu tiên reliability: từ "fix when broken" sang **quản lý burn rate chủ động**.

Metrics instrumentation tối thiểu

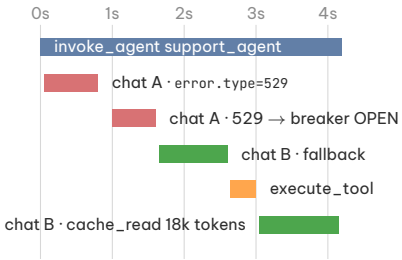
```python
from prometheus_client import Counter, Histogram, Gauge
# Counter(name, documentation, labelnames)
REQS = Counter("agent_requests_total", "LLM requests",
               ["provider", "status", "route"])
LAT = Histogram("agent_latency_seconds", "Latency",
                ["provider", "route"],
                buckets=(.25, .5, 1, 2.5, 5, 10, 30))
TOK = Counter("agent_tokens_total", "Tokens",
              ["provider", "kind"])   # input|cached|output
CACHE = Counter("cache_lookups_total", "Cache lookups",
                ["cache_type", "result"])     # hit|miss
CB = Gauge("circuit_state", "0 closed 1 open 2 half",
           ["provider"])

REQS.labels("B", "ok", "fallback:circuit_open").inc()
# NEVER use user_id / prompt as a label
# -> cardinality explosion
```

Report cần có

- Latency P50 / P95 / P99 (+ TTFT nếu stream)
- Availability theo lượt user, error rate theo loại lỗi
- Fallback success rate, **route reason**
- Cache hit rate + ví dụ false hit (gắn nhãn tay)
- Retry amplification = attempt / LLM call
- Recovery time trong chaos test

False hit cần chấm (người hoặc LLM-as-judge) → ghi vào report, không phải label realtime.



OTel GenAI semantic conventions

trạng thái: *Development*; repo riêng từ 06/2026

- Span: `gen_ai.operation.name`, `gen_ai.provider.name`, `gen_ai.request.model`, `error.type`
- Usage: `gen_ai.usage.input_tokens`, `…output_tokens`, `…cache_read.input_tokens`
- Metric: `gen_ai.client.operation.duration`
- Thuộc tính riêng: `app.route_reason`, `app.circuit_state`, `app.cache_result`

Langfuse, LangSmith, Arize Phoenix đều nhận OTLP → đổi backend không phải sửa code. Chấm 1–5% trace bằng LLM-as-judge → **quality SLI**.

Chaos testing: cố tình làm hỏng

Băng sự cố trong Lab 25B

- G1 Traffic ×2,5; 429 kèm `retry-after`
- G2 Primary mất 75% GPU (529, hàng đợi) — retry storm
- G3 Brown-out: 30% request lỗi 500
- G4 Chậm thêm 5 s, không có lỗi nào
- G5 Bug deploy (400), rồi hết quota
- G6 Provider A sập (503 + treo); B hạ rate limit
- C1 Mọi provider sập 2 phút; tài liệu nguồn đổi

Công cụ

- Lab: simulator discrete-event (thời gian ảo, seed cố định) — tự viết thêm băng bằng YAML
- Production: LiteLLM `mock_testing_fallbacks`, Toxiproxy (latency / timeout mạng), feature flag `CHAOS_*`; nên thêm ca Redis chết, cost cap gần cạn

Bằng chứng cần thấy (check của băng)

- Availability theo lượt user $\geq$ 95–98% (G5: $\geq$ 78%)
- Amplification (attempt / LLM call) $\leq$ 1,6 — naive ở G2: 9,9
- Recovery $\leq$ 30 s; $\geq$ 80% traffic quay lại primary
- 0 vi phạm `retry-after`, 0 retry lỗi 400, 0 tool call sang model không có tool
- Cache: 0 câu trả lời sai, FAQ vẫn trả lời được khi mọi provider sập

Lưu ý: Chạy ở staging / shadow traffic trước; giới hạn blast radius; luôn có **kill switch**. OpenAI (02/2026) và Google Cloud (06/2025) đều chỉ ra retry làm hồi phục chậm hơn — hãy test điều đó *trước* khi production test giúp bạn.

Team đặt **SLO nội bộ** availability $\geq$ 99% và ký **SLA** 99,5% với khách hàng. Nhận xét?

A. Hợp lý — SLA cao hơn để hấp dẫn khách hàng

B. SLO và SLA luôn phải bằng nhau

C. Ngược: SLO nội bộ phải **chặt hơn** SLA (vd. SLO 99,9%, SLA 99,5%) để có biên cảnh báo trước khi vi phạm hợp đồng

D. Đã có SLA thì không cần SLO

Cả lớp vote A / B / C / D — 1 phút

SLO availability **99,5%**, cửa sổ 30 ngày, ~2 triệu lượt request / tháng. Error budget là bao nhiêu?

A. 5% ≈ 36 giờ

B. 0,5% ≈ 3,6 giờ downtime, hay ≈ 10.000 request lỗi / 30 ngày

C. 0,05% ≈ 22 phút

D. 99,5% ≈ 716 giờ

Cả lớp vote A / B / C / D — 1 phút

SLO availability 99,5%. Alert nào nên **page** on-call ngay?

A. Error rate 7,2% suốt 1 giờ qua và vẫn cao trong 5 phút gần nhất (burn rate 14,4)

B. Có 1 request lỗi 500

C. Error rate 0,6% kéo dài 3 ngày

D. P95 tăng 5% so với tuần trước

Cả lớp vote A / B / C / D — 1 phút

Live Demo & Lab 25

07

Xem retry storm và breaker trên simulator — rồi vào vai on-call: sửa client, chứng minh bằng 7 băng sự cố

Live demo: Game Day trên simulator (10 phút)

LIVE DEMO

1. **Baseline**: repo Lab 25B, bản lời giải — `make compare`: client naive 0/6 băng, ResilientClient 6/6
2. **Retry storm**: băng G2 (`alpha` mất 75% GPU) + `make plot` — provider hồi phục ở t=300 nhưng naive vẫn 0% thành công tới t≈455 (amplification ≈ 10)
3. **Breaker + fallback**: cùng băng với ResilientClient — breaker OPEN → `alpha-eu` / `beta`; amplification 1,05, recovery 0 s
4. **Cache**: `make cache` (băng C1: mọi provider sập 2 phút) — FAQ availability 0,84 → 0,96 nhờ stale-if-error, cost / lượt −34%; ngưỡng 0,80 → 157 câu trả lời nhầm: chỉ ra 1 cặp "gần mà khác" rồi siết ngưỡng

LAB #25

Mục tiêu: Game Day (Lab 25B): thay client v1 gây retry storm bằng client chịu lỗi và chứng minh bằng số trên 6 băng sự cố + băng cache C1; thêm workflow exactly-once, burn-rate alert và postmortem

Deliverable: repo Lab 25B đã hoàn thành (code Part A–G, `configs/gameday.yaml`), `reports/REPORT.md` có postmortem G2, `reports/scoreboard.json` từ `make grade`; đưa các pattern vào reliability layer của project Milestone 2

`github.com/VinUni-AI20k/K4-Track03-Circuit-Breakers-Caching-Reliability`

Thời gian: 2 giờ trên lớp (core: A–C) + 2 giờ (G, D, E, F) — mọi phần đều được chấm

Thảo luận nhóm — 7 phút

Sau khi xem hình G2 của client naive, mỗi nhóm thiết kế **1 băng sự cố mới** cho ShopAgent (khác G1–G6, C1):

- Hỏng cái gì, ở deployment nào, từ giây nào tới giây nào? Client đúng phải phản ứng ra sao?
- Check nào (availability, amplification, recovery, p95…) chứng minh hệ thống chịu được — ngưỡng bao nhiêu?
- Nhóm khác phản biện: nếu chạy băng này trên production thật thay vì simulator, **side effect** với user / dữ liệu là gì? **Kill switch** là gì?

Nhóm 2–3 người · mời 1–2 nhóm chia sẻ

Lab 25 — Milestones (thiết kế 4 giờ)

Thời gian	Việc cần làm	Kiểm chứng
0–15’	Kickoff: `make baseline`, hình G2 của client naive; thảo luận D4	—
15–40’	**A**: phân loại lỗi (429 ≠ 429 hết quota ≠ 529 ≠ 400), full jitter, retry budget	unit test
40–80’	**B**: breaker sliding-window, slow call, giới hạn probe, permit / generation	unit test
80–115’	**C**: ResilientClient: ladder lọc tool calling, cooldown, failover (≥ 4/6 băng)	`make compare`
115–155’	**G**: semantic cache có scope, ngưỡng, TTL, invalidation, stale-if-error	`make cache`
155–185’	**D**: hoàn tiền exactly-once: idempotency key + checkpoint + resume	`make workflows`
185–215’	**E**: burn-rate alert đa cửa sổ + error budget 30 ngày	`make slo`
215–240’	**F**: report + postmortem G2 (hoàn thiện ở nhà); 2 nhóm chia sẻ	`REPORT.md`

Core rồi mở rộng

Core: 0–115’ (A–C). Làm **G ngay sau C** vì băng C1 dùng ResilientClient.

Metric, report & rubric chấm điểm

Metric (simulator in ra)

- availability (theo lượt user), P95
- amplification, retry ratio, cost / lượt
- recovery, traffic quay lại primary
- vi phạm `retry-after`, retry lỗi 400
- tool call sang model không có tool
- cache: hit ratio, FAQ availability, stale
- trả lời sai từ cache (false hit, lộ dữ liệu, prompt cũ, bỏ qua invalidation) và lỗi thời

Report bắt buộc

- bảng 6 băng naive vs của nhóm + config có lý do
- 3 thí nghiệm; workflow & alert; bảng cache
- postmortem G2 blameless có số liệu

Phần	Điểm	Chấm thế nào
A. Lỗi & retry	12	unit test
B. Circuit breaker	12	unit test
C. ResilientClient	30	test (6) + 6 băng × 4
G. Cache	10	test (4) + băng C1 (6)
D. Workflow hoàn tiền	12	test (4) + 500 workflow (8)
E. Alerting & budget	12	test (4) + SLI (4) + SLI ẩn (4)
F. Report	12	chấm tay; postmortem G2 = 5

Lưu ý: 88 điểm tự động (`make grade`) + 12 chấm tay. Băng chỉ có điểm khi PASS mọi check trên mọi seed của băng ẩn; hard-code thời điểm / tên băng = 0 điểm phần đó. Report không có số liệu: trừ tới 5 điểm; code không chạy trên máy sạch: −10.

Milestone 2 & demo cuối lab

Milestone 2 — nộp hôm nay

- Fine-tuned model (SFT / DPO)
- Guardrails + eval pipeline (Ngày 24)
- **Reliability patterns** (Lab 25)
- GitHub repo + portfolio + video demo 5 phút

Chia sẻ cuối lab (2 nhóm × 5 phút)

1. `make compare` + `make cache`: bao nhiêu băng PASS?
2. Postmortem G2: timeline từ CSV, trigger vs root cause, vì sao không tự hồi phục sau t=300
3. Băng C1: có / không cache; 1 cặp câu "gần mà khác" và ngưỡng đã chọn
4. 1 failure mode client của nhóm vẫn xử lý kém
5. 1 config sẽ đổi nếu deploy production

Lưu ý: Peer review chéo theo rubric: code quality, eval completeness, reliability — feedback phải actionable.

Tổng kết — Key Takeaways

Những ý chính cần nhớ trước khi sang bài tiếp theo

1. **Retry** chỉ cho lỗi transient, ở **1 tầng**, có jitter + retry budget
2. **Breaker + fallback ladder** (bậc 0: cùng model, khác region) là mức tối thiểu
3. **Cache** cần guardrail: prefix ổn định, semantic cache có scope, đo false hit
4. **SLO + error budget**, alert theo **burn rate**; quality SLO ngang uptime
5. Side effect: **idempotency + durable execution**; chaos test tạo bằng chứng

Tiếp theo & Bài tập

VINUNIVERSITY

Bài tiếp theo

Ngày 26: MCP — chuẩn hóa tool

"Reliability đã có — tiếp theo chuẩn hóa cách agent kết nối tool. MCP server cũng là một dependency: cần timeout, breaker và cache như LLM provider."

Bài tập về nhà

- Hoàn thành Lab 25 (Part A–G + report), chạy `make grade`, nộp Milestone 2
- Đọc: Brooker, "Exponential Backoff And Jitter"
- Đọc: SRE Workbook, "Alerting on SLOs"
- Đọc: Anthropic, "A postmortem of three recent issues"

Tài liệu tham khảo (cập nhật 10/2026)

Nền tảng

1. Nygard, M. (2018). *Release It!* (2nd ed.). Pragmatic Bookshelf.
2. Microsoft Azure Architecture Center. *Circuit Breaker pattern*. learn.microsoft.com/azure/architecture/patterns/circuit-breaker
3. Fowler, M. (2014). *CircuitBreaker*. martinfowler.com/bliki/CircuitBreaker.html
4. Brooker, M. (2015). *Exponential Backoff And Jitter*. AWS Architecture Blog.
5. Beyer et al. (2018). *The Site Reliability Workbook*, ch. 5 "Alerting on SLOs". sre.google/workbook/alerting-on-slos
6. Dean & Barroso (2013). *The Tail at Scale*. CACM 56(2).
7. Prometheus. *client_python* — Counter, Gauge, Histogram.

Cập nhật 2025–2026

8. Anthropic (09/2025). *A postmortem of three recent issues*.
9. Google Cloud (06/2025) incident report; OpenAI status write-up (02/2026).
10. LiteLLM docs: Router — fallbacks, retries, cooldowns; Security update (03/2026). docs.litellm.ai
11. Prompt caching docs: Anthropic, OpenAI, Google Gemini (truy cập 10/2026).
12. *vCache: Verified Semantic Prompt Caching*. arXiv:2502.03771 (ICLR 2026).
13. *Don't Break the Cache*. arXiv:2601.06007 (01/2026).
14. OpenTelemetry. *Semantic conventions for GenAI*. github.com/open-telemetry/semantic-conventions-genai
15. Datadog (04/2026). *State of AI Engineering*. datadoghq.com/state-of-ai-engineering
16. Langfuse docs: observability, traces, cost & latency metrics.

Hỏi & Đáp

Circuit breaker nên đặt ở đâu: trong từng service hay tập trung tại gateway?

Hỏi & Đáp — Câu hỏi mở (1/2) THẢO LUẬN

VINUNIVERSITY

Q1. Circuit breaker nên đặt ở đâu: trong từng service hay tập trung tại gateway?

Q2. Retry, circuit breaker và fallback khác nhau thế nào? Dùng theo thứ tự nào?

Q3. Có nên dùng multi-provider ngay từ MVP? Chi phí ẩn là gì?

Q4. Semantic cache có làm mất tính cá nhân hóa của agent không?

Chọn 2–3 câu để thảo luận cả lớp

Hỏi & Đáp — Câu hỏi mở (2/2)

THẢO LUẬN

VINUNIVERSITY

Q5. Làm sao test fallback mà không phải chờ provider sập thật?

Q6. Agent chạy 50 bước, provider lỗi ở bước 37 — làm sao không phải chạy lại từ đầu?

Q7. Prompt caching và semantic caching dùng cùng lúc được không?

Q8. Reliability của agent khác gì reliability của web service truyền thống?

Gợi ý trả lời: xem file đáp án dành cho giảng viên



Cảm ơn!

AICB-P2T3 · Ngày 25 · Circuit Breakers, Caching & Reliability

github.com/vinuni-aicb

Liên hệ: instructor@vinuni.edu.vn